

HW 2-B: OWASP Juice Shop / OWASP Top 10

Course: CSCE 703

UIN: 937006978

Email: srijapentyala@tamu.edu

GitHub: <https://github.com/srijapentyala/hw2b-juice-shop-login>

Part 1: Secure feature design

I ran Juice Shop locally and designed registration from three bugs.

1. Login builds its query by concatenating the email. I typed ' OR 1=1-- and a junk password and was signed in as admin@juice-sh.op. Juice Shop awarded Login Admin. A registration insert must use a parameterized query, so the email stays data and cannot add an administrator row.

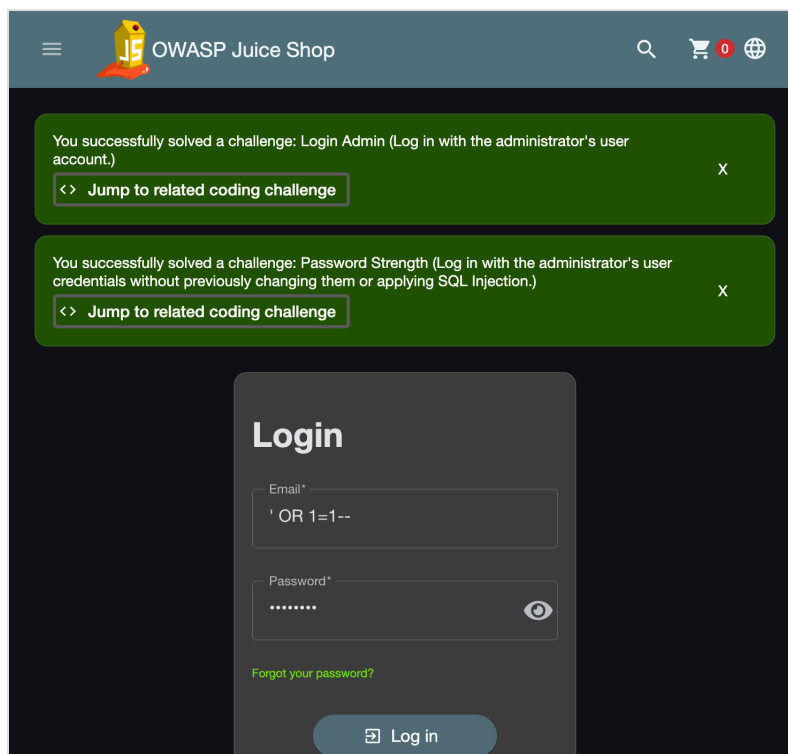


Figure 1. The injected email is still in the form. Juice Shop awarded Login Admin for that login, and Password Strength for the real administrator password admin123.

2. Search writes the query into the page as HTML. My term rendered as a yellow XSS heading instead of text. Encode a registered username on output, and send a Content-Security-Policy that blocks inline script, so a tag in the name is shown and does not run.

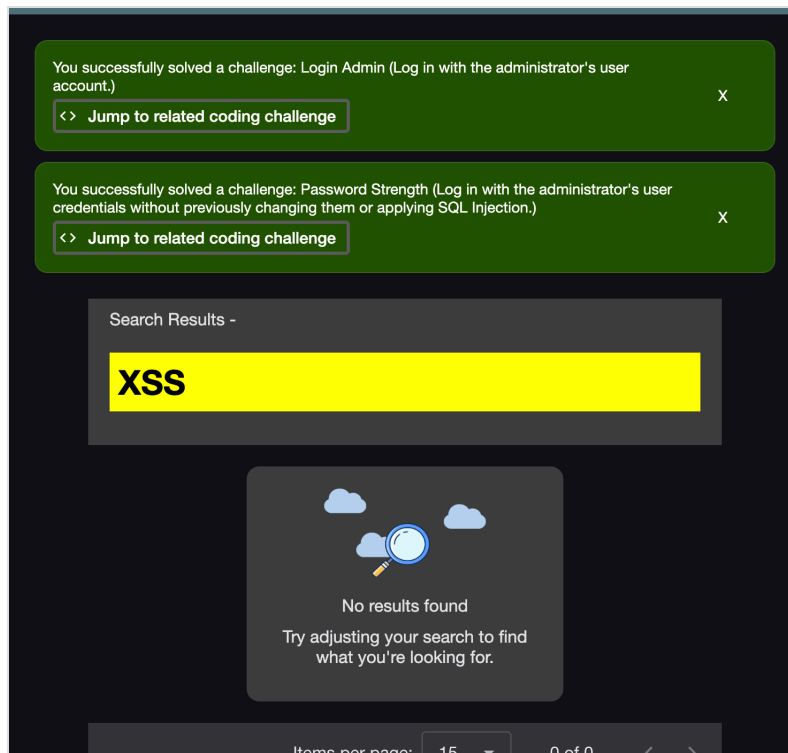


Figure 2. The search term was parsed as HTML. The tags never appear as text.

3. The administrator password is admin123, and the login token includes its MD5 hash. Password Strength solved only when I used that password, not injection. Registration should reject common passwords, store a bcrypt hash with a unique salt and cost 12, and never return the hash.

```
bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
bcrypt.checkpw(password.encode(), stored_hash)
```

Part 2: Front-end login form

I built a login page with Juice Shop's two fields, email and password.

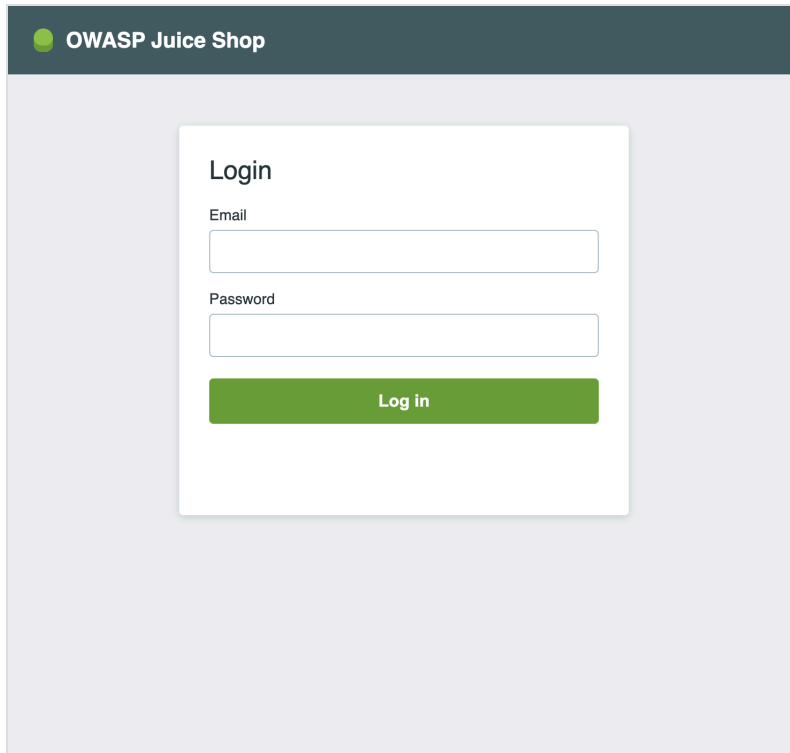
1. `login.js` blocks an empty submit, requires "@" in the email, and requires at least 8 characters. Those errors stay in the browser.

2. A form that passes is sent as JSON to `POST /login`.

3. `server.py` repeats the same three checks, then compares the password with a PBKDF2 hash in constant time. A wrong password and an unknown email get the same message. Nothing is placed in a SQL string.

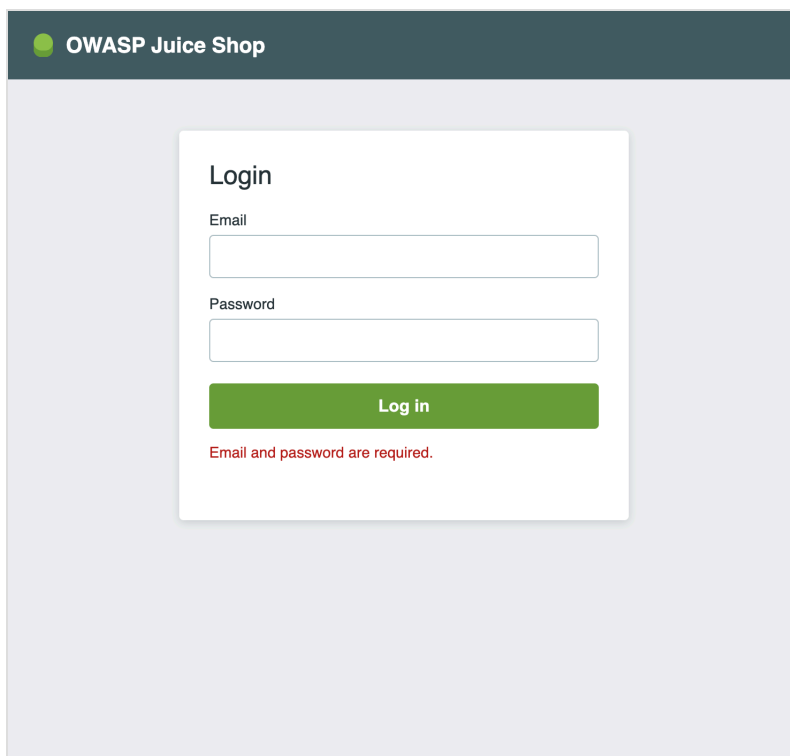
Run `python3 server.py` and open `http://127.0.0.1:8080`. The demo account is `demo@juice-sh.op / JuiceShop1`.

Repository: <https://github.com/srijapentyala/hw2b-juice-shop-login>



The image shows the OWASP Juice Shop login page. At the top is a dark blue header with the OWASP Juice Shop logo and name. Below the header is a light gray background. In the center is a white login form with the title "Login". The form contains two input fields: "Email" and "Password". Below the input fields is a green "Log in" button.

Figure 3. Login form before submission.



The image shows the OWASP Juice Shop login page after a client-side validation check. The form is the same as in Figure 3, but now there is a red error message below the "Log in" button that reads "Email and password are required.".

Figure 4. Client-side check rejects an empty submission.

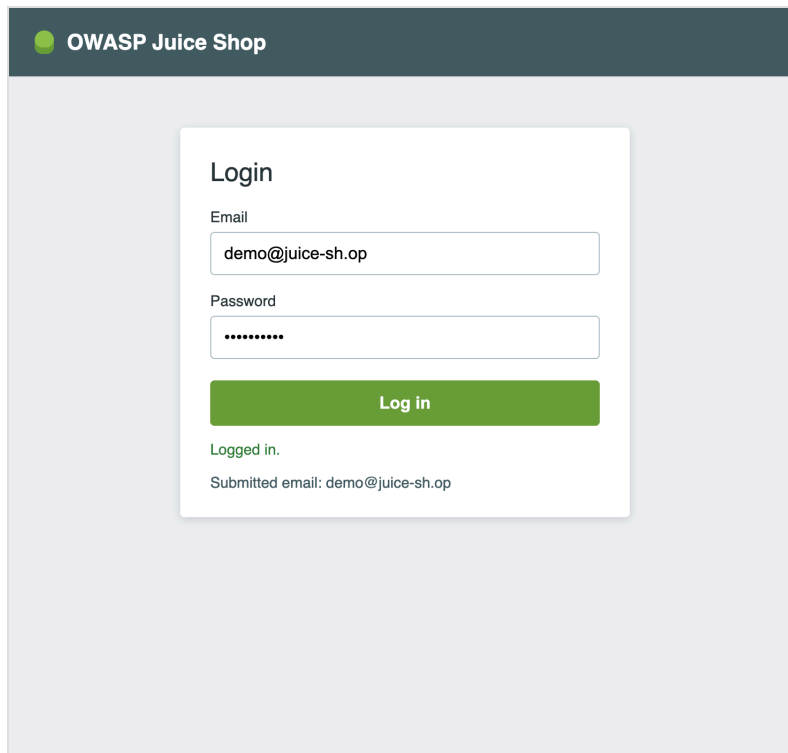


Figure 5. Demo account passes both the browser check and the server check.

Part 3: Trying to break the form

I repeated the Juice Shop attacks on `http://127.0.0.1:8080` after `python3 server.py`.

1. SQL injection. I entered ' OR 1=1--@juice-sh.op and password1, then clicked Log in. The format checks passed. The server answered "Invalid email or password." There is no SQL string to rewrite, so the login failed.

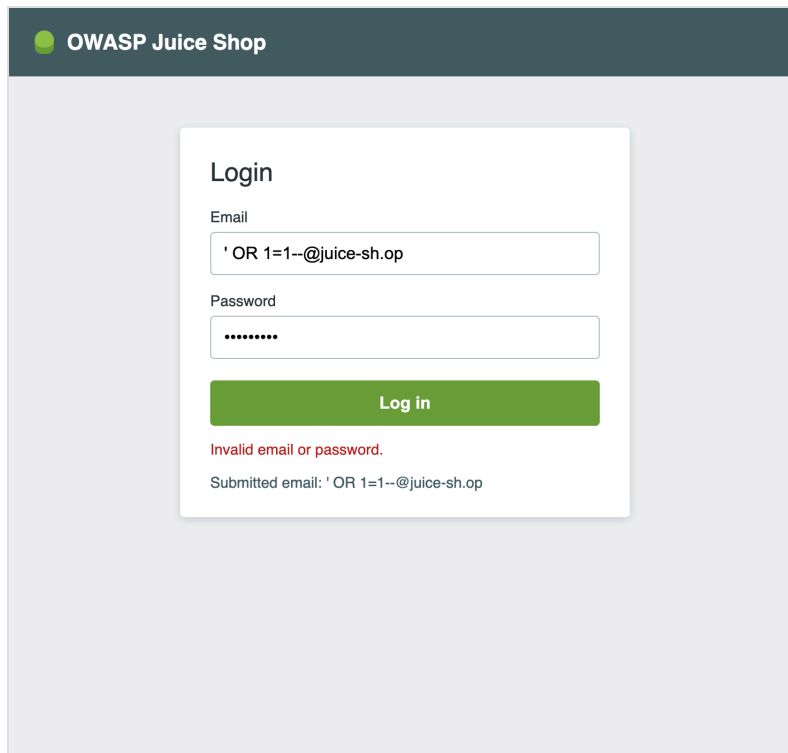


Figure 6. The SQL fragment is stored as an email address. Login fails.

2. XSS, which succeeds on this repository. `login.js` writes the email with `innerHTML`. I entered `<img src=x onerror=document.getElementById('message').textContent='JavaScript executed'>@a.co` and `password1`, then clicked Log in. The image failed, the handler ran, and the status line became "JavaScript executed." The login itself still failed.

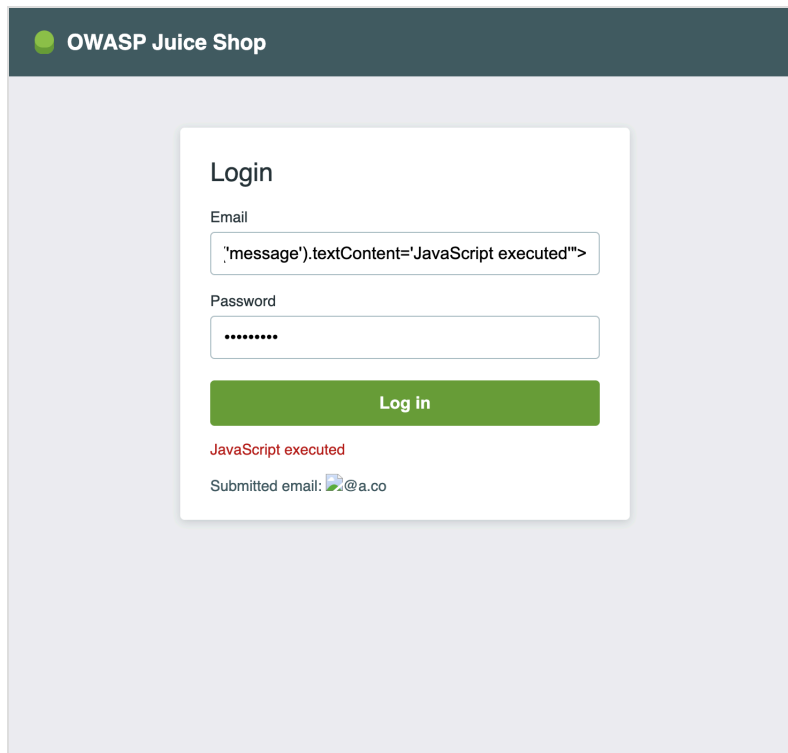


Figure 7. The status line was written by the image handler, not by the login code.

Fix: assign that line with `textContent`, and send `Content-Security-Policy: script-src 'self'`, so the email is text and an inline handler cannot run.